5.2 Snowbridge (이더리움 브릿지) 구현 분석

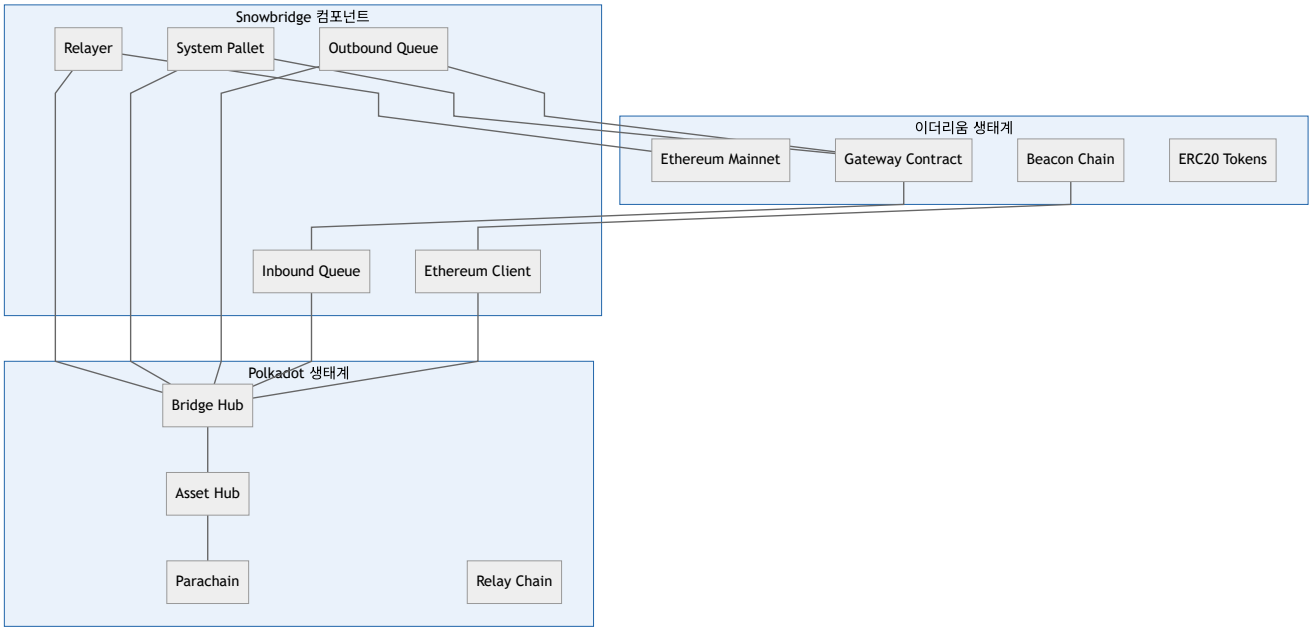
개요

Snowbridge는 Polkadot과 이더리움 간의 신뢰할 수 있는 브릿지로, 이더리움의 비콘 체인(Beacon Chain) 합의를 추적하는 온체인 라이트 클라이언트를 기반으로 합니다. 이 브릿지는 토큰 전송뿐만 아니라 일반적인 메시지 전달과 트랜잭션 실행을 지원하는 범용 브릿지입니다.

Snowbridge는 두 가지 주요 버전으로 구성되어 있으며, V2는 수수료 절감, 사용자 경험 개선, 릴레이어 분산화 향상, 그리고 “트랜잭팅” 기능을 통한 범용 브릿지 기능을 제공합니다.

Snowbridge 아키텍처 및 구성 요소

5.2.1 전체 아키텍처 개요



5.2.2 핵심 팔렛 구조

Snowbridge는 다음과 같은 핵심 팔렛들로 구성됩니다:

```
snowbridge/pallets/
├── ethereum-client/ # 이더리움 비콘 체인 라이트 클라이언트
├── system/ # 브릿지 시스템 관리 및 거버넌스
├── inbound-queue/ # 이더리움 → Polkadot 메시지 처리
```

```
├── outbound-queue/ # Polkadot → 이더리움 메시지 처리
├── system-frontend/ # 시스템 프론트엔드 인터페이스
├── inbound-queue-v2/ # 인바운드 큐 V2 구현
├── outbound-queue-v2/ # 아웃바운드 큐 V2 구현
└── system-v2/ # 시스템 팔렛 V2 구현
```

이더리움-Polkadot 간 자산 전송 메커니즘

5.2.3 Polkadot → 이더리움 전송 프로세스 (V2)

1단계: 사용자 에이전트가 초기 XCM 구성

사용자 에이전트는 소스 파라체인 S에서 실행될 초기 XCM 메시지 x_0를 구성합니다:

```
// 예시: KILT에서 이더리움으로 토큰 전송
WithdrawAsset (KLT, 100)
PayFees (KLT, 20)
InitiateAssetsTransfer asset=(KLT, 60) remoteFee=(KLT, 20) dest=AH
ExchangeAsset give=(KLT, 20) want=(WETH, 1)
InitiateAssetsTransfer asset=(KLT, 40) remoteFee=(WETH, 1) dest=Ethereum
DepositAsset (KLT, 40) beneficiary=Bob
```

2단계: 수수료 추정

사용자 에이전트는 다음 공식을 사용하여 총 비용을 계산합니다:

$$

\beta \left(a + \frac{b + c}{K_{P'/S'}} + \frac{d + r}{K_{E'/S'}}\right)

$$

여기서:

- a: 소스 체인에서의 실행 비용 (S' 토큰)
- b, c: Asset Hub와 Bridge Hub에서의 실행 비용 (DOT)
- d: 이더리움에서의 실행 비용 (ETH)
- r: 릴레이어 보상 (ETH)
- β: 환율 변동성을 고려한 승수
- K: 환율

3단계: Asset Hub에서 메시지 처리

Asset Hub는 다음과 같은 XCM을 실행합니다:

```
ReserveAssetDeposited (KLT, 80)

PayFees (KLT, 20)

SetAssetClaimer Kilt/Alice

AliasOrigin Kilt/Alice

ExchangeAsset give=(KLT, 20) want=(WETH, 1)

InitiateAssetsTransfer asset=(KLT, 60) remoteFee=(WETH, 1) dest=Ethereum

DepositAsset (KLT, 60) beneficiary=Bob
```

4단계: Bridge Hub에서 메시지 처리

Bridge Hub의 `SnowbridgeMessageExporter` 가 메시지를 처리하여:

- 브릿지 명령 m을 바이너리 머클 트리 M_n에 커밋
- 릴레이어 보상 r을 스토리지에 설정: P(hash(m)) = r

5.2.4 이더리움 → Polkadot 전송 프로세스 (V2)

1단계: Gateway 컨트랙트에서 전송 시작

이더리움의 Gateway 컨트랙트는 다음과 같은 인터페이스를 제공합니다:

```
enum AssetTransferType {
ReserveDeposit, ReserveWithdraw, Teleport
}

struct Token {
AssetTransferType type;
address token;
uint128 amount;
}

function send(
ParaID destinationChain,
Beneficiary calldata beneficiary,
Token[] tokens,
uint128 reward
) external payable;
```

2단계: 릴레이어가 메시지를 Bridge Hub로 전달

릴레이어는 이더리움에서 발생한 메시지를 Bridge Hub로 전달하며, 이 과정에서 다음 XCM이 생성됩니다:

```
WithdrawAsset (KLT, 100)

ReserveAssetDeposited(WETH, 2)

PayFees (WETH, 1)

SetAssetClaimer Kilt/Bob

AliasOrigin Ethereum/Alice

InitiateAssetsTransfer asset=(KLT, 100) remoteFee=(WETH, 1) dest=KLT

DepositAsset (KLT, 100) beneficiary=Bob
```

스마트 컨트랙트 및 팔렛 구현 분석

5.2.5 Ethereum Client 팔렛

목적: 이더리움 비콘 체인의 합의 업데이트를 검증하는 라이트 클라이언트

핵심 기능:

- 동기화 위원회가 서명한 합의 업데이트 검증
- 최종화된 비콘 헤더 추적
- 동기화 위원회 업데이트 처리

주요 엔트리포인트:

```
// 합의 업데이트 제출

pub fn submit(

origin: OriginFor<T>,

update: Box<Update>

) -> DispatchResultWithPostInfo
```

```
// 강제 체크포인트 설정 (거버넌스)
pub fn force_checkpoint(
origin: OriginFor<T>,
update: Box<CheckpointUpdate>
) -> DispatchResult


// 운영 모드 설정
pub fn set_operating_mode(
origin: OriginFor<T>,
mode: BasicOperatingMode
) -> DispatchResult
```

검증 프로세스:

1. BLS 서명 검증을 통한 동기화 위원회 합의 확인
2. 머클 증명을 통한 비콘 상태 검증
3. 포크 버전 및 도메인 검증
4. 최종성 확인 및 헤더 저장

5.2.6 System 팔렛

목적: 브릿지의 거버넌스 API 및 이더리움 측 제어

핵심 기능:

- Gateway 컨트랙트 업그레이드

- 운영 모드 업데이트
- 토큰 등록 및 관리
- 에이전트 및 채널 생성

주요 엔트리포인트:

```
// 토큰 등록
pub fn register_token(
origin: OriginFor<T>,
location: Box<VersionedLocation>,
metadata: AssetMetadata
) -> DispatchResult

// Gateway 업그레이드
pub fn upgrade(
origin: OriginFor<T>,
impl_address: H160,
impl_code_hash: H256,
initializer: Option<Initializer>
) -> DispatchResult

// 운영 모드 설정
pub fn set_operating_mode(
origin: OriginFor<T>,
```

```
 mode: OperatingMode

 ) -> DispatchResult
```

5.2.7 Inbound Queue 팔렛

목적: 이더리움에서 Polkadot으로의 메시지 처리

핵심 기능:

- 이더리움 메시지 읽기 및 검증
- XCM을 통한 목적지 전달
- 메시지 실행 결과 추적

메시지 처리 플로우:

1. 이더리움 로그에서 메시지 추출
2. 머클 증명을 통한 메시지 검증
3. XCM 메시지로 변환
4. 목적지 파라체인으로 전달

5.2.8 Outbound Queue 팔렛

목적: Polkadot에서 이더리움으로의 메시지 전송

핵심 기능:

- XCM 메시지를 이더리움 명령으로 변환
- 머클 트리를 통한 메시지 커밋
- 릴레이어 보상 추적

메시지 처리 플로우:

1. XCM 메시지 수신 및 파싱

2. 이더리움 명령으로 변환

3. 머클 트리에 커밋

4. 릴레이어 보상 설정

Snowbridge 배포 및 운영 가이드

5.2.9 배포 아키텍처

이더리움 측 컴포넌트:

- Gateway 컨트랙트: 메시지 전송 및 수신의 진입점
- BeaconLightClient 컨트랙트: Polkadot 상태 검증
- ERC20 토큰 컨트랙트: 브릿지된 자산 관리

Polkadot 측 컴포넌트:

- Bridge Hub 파라체인: Snowbridge 팔렛들이 배포된 전용 파라체인
- Asset Hub 연동: 자산 관리 및 XCM 라우팅
- 릴레이어 서비스: 오프체인 메시지 전달

5.2.10 릴레이어 시스템

릴레이어 역할:

1. **합의 릴레이어**: 이더리움 비콘 체인 업데이트를 Bridge Hub로 전달
2. **메시지 릴레이어**: 양방향 메시지 전달 처리
3. **증명 릴레이어**: 전달 증명 제출 및 보상 청구

릴레이어 보상 시스템:

- 모든 보상은 Bridge Hub에서 WETH로 누적
- 수동 청구 방식으로 보상 지급

- 세금 회계 목적으로 보상 계정과 수령 계정 분리 가능

```
// 보상 청구 함수
claim(rewards_account: AccountId, withdrawal_account: AccountId)
```

5.2.11 운영 모니터링

핵심 메트릭:

- 비콘 헤더 동기화 지연시간
- 메시지 전달 성공률
- 릴레이어 활성도
- 가스 비용 및 수수료 추이

알림 시스템:

- 합의 업데이트 실패
- 메시지 전달 지연
- 릴레이어 오프라인
- 비정상적인 수수료 급증

5.2.12 보안 고려사항

라이트 클라이언트 보안:

- 동기화 위원회의 2/3 이상 합의 요구
- 포크 선택 규칙 준수
- 장기간 공격(Long Range Attack) 방지

메시지 검증:

- 머클 증명을 통한 메시지 무결성 보장
- 재생 공격(Replay Attack) 방지
- 순서 보장 및 중복 처리 방지

경제적 보안:

- 릴레이어 보상을 통한 인센티브 정렬
- 최소 보상 요구사항으로 스팸 방지
- 동적 수수료 조정 메커니즘

5.2.13 업그레이드 및 거버넌스

거버넌스 채널:

- Primary Governance Channel: 핵심 시스템 업그레이드
- Secondary Governance Channel: 운영 매개변수 조정

업그레이드 프로세스:

1. Polkadot 거버넌스를 통한 제안 및 투표
2. Bridge Hub에서 업그레이드 명령 생성
3. 릴레이어를 통한 이더리움 Gateway 업그레이드
4. 새로운 구현체 활성화

비상 대응:

- 운영 모드 전환 (정상/제한/중단)
- 긴급 중단 메커니즘
- 자산 복구 절차

이 문서는 Snowbridge의 전체적인 아키텍처와 구현 세부사항을 다뤘습니다. 다음 문서에서는 커스텀 브릿지 개발을 위한 가이드라인과 베스트 프랙티스를 다룰 예정입니다.